



THE CATHOLIC UNIVERSITY OF AMERICA

DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

EE 516 Power Systems

Project 1: Simulation and Analysis of a Nonlinear Electromagnet

Submitted to: Georges Nehmetallah(Phd)

Student Name: Mulualet Ayana

Submitted On: 19 Mar 2026

Table of Contents

1. Introduction
2. Project Overview and Simulations
3. How the MATLAB Code Works
4. How the Simulink Model Works
5. Comparing MATLAB and Simulink Implementations
6. Power and Energy Analysis
7. Conclusions
8. Appendix/Source Code

1. Introduction

This project investigates the behavior of an electromagnet with a moving part. The system consists of a stationary magnetic core with a coil, and a movable member attached to a spring and damper. When voltage is applied to the coil, magnetic force pulls the movable member toward the core. The closer it gets, the stronger the pull becomes this is what makes the system nonlinear.

The objectives of this project are:

1. Derive the state equations describing the system dynamics from first principles
2. Implement numerical simulations using MATLAB's ode45 solver
3. Build a hierarchical Simulink model for the same system
4. Run three distinct simulations to analyze system behavior under different conditions
5. Validate steady-state operating conditions using theoretical equations
6. Compute and verify power and energy conservation
7. Compare and contrast MATLAB and Simulink implementations

The project uses two simulation methods:

- **MATLAB programming** using the ode45 solver
- **Simulink modeling** with hierarchical subsystems

Both methods use the same physical parameters and equations, ensuring consistent results.

System Parameters:

- Resistance: $r = 10 \, \Omega$
- Inductance coefficient: $k = 6.283 \times 10^{-5} \, \text{H} \cdot \text{m}$
- Initial position: $x_0 = 3 \, \text{mm}$
- Spring constant: $K = 2667 \, \text{N/m}$
- Damping coefficient: $D = 4 \, \text{N} \cdot \text{s/m}$
- Mass: $M = 0.055 \, \text{kg}$

Initial conditions for all simulations:

- Current: $i = 0 \, \text{A}$
- Position: $x = 3 \, \text{mm}$
- Velocity: $v = 0 \, \text{m/s}$

2. Project Overview and Simulations

Simulation 1: Fixed position at 3 mm, step voltage of 5V, its purpose is to isolate electrical response

Simulation 2: Let the movable part move freely and applied 5 volts, its purpose is to show how electrical and mechanical systems interact

Simulation 3: Applied a 5V pulse (0.05-0.15s) and a 4N force pulse (0.2-0.3s), its purpose is to show realistic operation with timed inputs

Simulation 1: Steady-State

In the Steady-State simulation, the movable member is held fixed at 3 mm. A 5V step is applied at $t=0$.

What happens?

- Current rises exponentially from 0 to 0.5 A with time constant $\tau = L/r = 2.09$ ms
- At steady-state: $i = v/r = 5/10 = 0.5$ A
- Magnetic force: $f_e = -k \cdot i^2 / (2x^2) = -0.873$ N
- Position remains constant at 3 mm
- Velocity remains zero

This match's standard RL circuit theory, confirming the electrical model is correct.

Simulation 2: Free-moving Magnet

The magnet is free to move. A 5V step is applied at $t=0$ with no external force.

What happens?

- Current rises but is affected by motion
- Magnetic force pulls the member toward the core
- Position decreases from 3 mm to approximately 2.5 mm
- Inductance increases as position decreases ($L = k/x$)
- At steady-state: $i = 0.5$ A, $x = 2.5$ mm, $f_e = -4.5$ N
- The magnetic pull balances the spring force: $k \cdot i^2 / (2x^2) = K(x - x_0)$

Simulation 3: Sub-Layer Version

Timed pulses are applied:

- **Voltage:** 5V from 0.05s to 0.15s
- **Force:** 4N from 0.2s to 0.3s

3. How the MATLAB Code Works

Code Structure

The MATLAB code has several sections:

1. Parameter Definition

matlab

```
r = 10; k = 6.283e-5; x0 = 0.003; K = 2667; D = 4; M = 0.055;
```

Purpose: Defines all system parameters.

2. Input Functions

matlab

```
v_func = @(t) 5 * (t >= 0.05 & t < 0.15); % Voltage pulse
```

```
fm_func = @(t) 4 * (t >= 0.2 & t < 0.3); % Force pulse
```

Purpose: Define time-varying inputs.

3. System Equations ($y(1)=i$, $y(2)=x$, $y(3)=v_x$)

matlab

```
system_ode = @(t, y) [
```

```
    (y(2)/k) * (v_func(t) - r*y(1)) + (y(1)*y(3))/y(2); % di/dt
```

```
    y(3); % dx/dt
```

```
    (1/M) * (fm_func(t) - k*y(1)^2/(2*y(2)^2) - D*y(3) - K*(y(2) - x0)) % dv/dt
```

```
];
```

Purpose: defines all three state equations.

4. Simulation

matlab

```
[t, y] = ode45(system_ode, [0 0.5], [0; x0; 0], options);
```

Purpose: Configure solver accuracy and run simulation.

5. Post-processing

matlab

```
i = y(:,1); x = y(:,2); v_x = y(:,3);
```

```
f_e = -k * i.^2 ./ (2 * x.^2);
```

```
L = k ./ x;
```

```
p_source = v_func(t) .* i;
```

```
p_resistor = r * i.^2
```

Purpose: Calculate derived quantities, Calculate power.

4. How the Simulink Model Works

What Makes Simulink Different

While MATLAB uses text-based programming, Simulink uses a visual approach. Each block performs a specific operation:

- **Gain blocks** multiply signals by constants
- **Sum blocks** add or subtract signals
- **Integrator blocks** integrate signals over time
- **Math Function blocks** perform operations like squaring
- **Product blocks** multiply two signals
- **Scope blocks** display results graphically

The Hierarchical Structure

The Sub-Layer Version (Top Level) → the Circuit-Sub- Model → The Magnet Sub-Model → pulse_generator (voltage input) → pulse_generator1 (force input) → signals: current, position, velocity, fe

Internal blocks and their functions:

1. **Subtract block:** Computes $v - r \cdot i$ — the voltage available to change current after accounting for resistive drop
2. **Gain block with value (x/k):** Multiplies the available voltage by (x/k) — this is $1/L$ where $L = k/x$
3. **Product block:** Computes $i \cdot v_x$ for the motion effect
4. **Divide block:** Divides $i \cdot v_x$ by x to get $(i \cdot v_x)/x$ — the back EMF term
5. **Sum block:** Adds the electrical effect and the motion effect to get total di/dt
6. **Integrator block:** Integrates di/dt over time to get actual current i
7. **Output port:** Sends current out to the magnet subsystem

Signal flow of TheCircuit_SubModel:

1. Voltage v and feedback current i go to Subtract → $v - r \cdot i$
2. $v - r \cdot i$ multiplies by x/k → $(x/k)(v - r \cdot i)$
3. Current i and velocity v_x multiply → $i \cdot v_x$
4. $i \cdot v_x$ divides by x → $(i \cdot v_x)/x$
5. Sum block adds both terms → di/dt
6. Integrator converts di/dt to i
7. Current i sent to output port

The Magnet_SubModel Blocks

Block	Purpose	Calculation
Math Function (u ²)	i^2	Square current
Gain (-k/2)	$-k \cdot i^2/2$	Scale for force numerator
Divide	$(-k \cdot i^2)/(2x^2)$	Force f_e
Sum	$f_m + f_e - D \cdot v - K(x-x_0)$	Sum forces
Gain (1/M)	$(1/M)$	Convert force to acceleration
Integrator1	$\int a \, dt = v$	Velocity
Integrator2	$\int v \, dt = x$	Position

5. Comparing MATLAB and Simulink Implementations

Fundamental Similarities

Both implementations are built on exactly the same mathematical foundation:

Aspect	MATLAB	Simulink
Equations	Same differential equations	Same equations implemented in blocks
Parameters	Same numerical values	Same numerical values
Solver	ode45 with same tolerances	ode45 with same settings
Results	Identical numerical outputs	Identical numerical outputs

Key Differences

Aspect	MATLAB	Simulink
Representation	Text-based code	Visual block diagrams
Development approach	Type equations	Drag and connect blocks
Debugging	Breakpoints, variable inspection	Signal tracing, scopes

6. Power and Energy Analysis

6.1 Power Calculations

Quantity	Formula	Physical Meaning
Input power	$p_{in} = v \cdot i$	Power from voltage source
Resistive loss	$p_R = r \cdot i^2$	Power dissipated as heat
Magnetic field power	$p_{field} = d(\frac{1}{2}Li^2)/dt$	Rate of change of field energy
Mechanical power	$p_{mech} = -f_e \cdot v_x$	Power transferred to mechanical system

6.2 Energy Storage

Form	Formula
Magnetic field energy	$W_{field} = \frac{1}{2}Li^2$
Spring potential energy	$W_{spring} = \frac{1}{2}K(x-x_0)^2$
Kinetic energy	$W_{kinetic} = \frac{1}{2}Mv_x^2$

7. Conclusions

This project successfully simulated a nonlinear electromagnet using both MATLAB and Simulink.

Key Findings:

1. **System behavior** shows five distinct phases in response to timed inputs:

Rest (0-0.05s): $i=0$, $x=3\text{mm}$

- Voltage pulse (0.05-0.15s): i rises, x decreases to 2.5mm
- Coasting (0.15-0.2s): i decays, x returns to 3mm
- Force pulse (0.2-0.3s): x increases to 4.5mm
- Return (0.3-0.5s): x returns to 3mm

2. **Electromagnetic force** is always attractive and depends nonlinearly on both current and position ($f_e \propto i^2/x^2$).
3. **Inductance** varies with position ($L = k/x$), increasing as the gap decreases.
4. **Energy** transforms between electrical, magnetic, and mechanical forms while being conserved overall.
5. **Steady-states** were validated against theoretical equations, confirming simulation accuracy.
6. **MATLAB and Simulink** produced identical results, cross-validating both implementations.

8) Appendix/Source Code

1. Steadystate.m

```
%EE516 Power Systems
% Mulualem Ayena
%Steady state
r = 10;           % Ohms
k = 6.283e-5;     % H*m
x0 = 3e-3;        % Initial/held position [m] (3 mm)
K = 2667;         % Spring constant [N/m]
D = 4;           % Damping coefficient [N.s/m]
M = 0.055;        % Mass [kg]
v = @(t) 5;       % constant 5 V for t > 0
```

```
odefun = @(t, i) (x0/k) * (v(t) - r*i);
```

```
i0 = 0;
```

```
tspan = [0 0.5];
```

```
[t, i] = ode45(odefun, tspan, i0);
```

```
fe = -(k .* i.^2) ./ (2 * x0^2);
```

```
figure;
subplot(2,1,1)
plot(t, i, 'g', 'LineWidth', 1)
grid on
xlabel('Time (s)')
ylabel('Current (A)')
title('Current')
```

```
subplot(2,1,2)
plot(t, fe, 'b', 'LineWidth', 1)
grid on
xlabel('Time (s)')
ylabel('Force (N)')
title('Electromagnetic Force')
```

2. electromagnet.m

```
%EE516 Power Systems
% Mulualem Ayena
%elctromagnet
r = 10;           % Coil resistance [Ohms]
k = 6.283e-5;     % Inductance coefficient [H.m]
x0 = 0.003;       % Initial position/spring reference [m] (3 mm)
K = 2667;         % Spring constant [N/m]
D = 4;           % Damping coefficient [N.s/m]
M = 0.055;        % Mass [kg]
```

```
% Voltage profile: 5V from 0 to 0.3s, then 0V
```

```
v_func = @(t) 5 * (t >= 0 & t < 0.3);
```

```
% Force profile: 4N from 0.15 to 0.3s, otherwise 0
```

```

fm_func = @(t) 4 * (t >= 0.15 & t < 0.3);

%% SYSTEM ODEs
% State vector: y(1) = i (current), y(2) = x (position), y(3) = v_x (velocity)
system_ode = @(t, y) [
    % di/dt = (x/k)*(v - r*i) + (i*v_x)/x
    (y(2)/k) * (v_func(t) - r*y(1)) + (y(1)*y(3))/y(2);

    % dx/dt = v_x
    y(3);

    % dv_x/dt = (1/M)*(f_m + f_e - D*v_x - K*(x - x0))
    % where f_e = -k*i^2/(2*x^2)
    (1/M) * (fm_func(t) - k*y(1)^2/(2*y(2)^2) - D*y(3) - K*(y(2) - x0))
];

%% SIMULATION SETTINGS
t_start = 0;
t_end = 0.5;
t_span = [t_start t_end];

% Initial conditions
i0 = 0; % Initial current [A]
x_init = 0.003; % Initial position [m] (3 mm)
v_x0 = 0; % Initial velocity [m/s]
y0 = [i0; x_init; v_x0];

% Solve using ode45 with high resolution
options = odeset('RelTol', 1e-6, 'AbsTol', 1e-8, 'MaxStep', 0.0005);
[t, y] = ode45(system_ode, t_span, y0, options);

i = y(:, 1); % Current [A]
x = y(:, 2); % Position [m]
v_x = y(:, 3); % Velocity [m/s]

% Calculate derived quantities
f_e = -k * i.^2 ./ (2 * x.^2); % Electromagnetic force [N]
L = k ./ x; % Inductance [H]

% Calculate power quantities
p_source = v_func(t) .* i; % Instantaneous input power
p_resistor = r * i.^2; % Power dissipated in resistor
p_field = gradient(0.5 * L .* i.^2, t); % Power into magnetic field
p_mech = -f_e .* v_x; % Mechanical power

% Energy calculations
W_field = 0.5 * L .* i.^2 * 1000; % Magnetic field energy [mJ]
W_spring = 0.5 * K * (x - x0).^2 * 1000; % Spring energy [mJ]
W_kinetic = 0.5 * M * v_x.^2 * 1000; % Kinetic energy [mJ]

%% PLOT 1: Combined Main Results
figure('Position', [100 100 1400 1000]);

% Current
subplot(2,2,1);
plot(t, i, 'g-', 'LineWidth', 1);
hold on;
xlabel('time (seconds)', 'FontSize', 12);
ylabel('Current (A)', 'FontSize', 12);
title('(a) Current', 'FontSize', 14);

```

```

grid on;
xlim([0 0.5]);
ylim([0 0.6]);

% Position
subplot(2,2,2);
plot(t, x*1000, 'b-', 'LineWidth', 1);
hold on;
line([0 0.5], [3 3], 'Color', 'k', 'LineStyle', ':');
xlabel('time (seconds)', 'FontSize', 12);
ylabel('Position (mm)', 'FontSize', 12);
title('(b) Position', 'FontSize', 14);
grid on;
xlim([0 0.5]);
ylim([2.0 6.0]);

% Velocity
subplot(2,2,3);
plot(t, v_x*1000, 'g-', 'LineWidth', 1);
hold on;
line([0 0.5], [0 0], 'Color', 'k', 'LineStyle', ':');
xlabel('time (seconds)', 'FontSize', 12);
ylabel('Velocity (mm/s)', 'FontSize', 12);
title('(c) Velocity', 'FontSize', 14);
grid on;
xlim([0 0.5]);
ylim([-300 300]);

% Electromagnetic Force
subplot(2,2,4);
plot(t, f_e, 'c-', 'LineWidth', 1);
hold on;
line([0 0.5], [0 0], 'Color', 'k', 'LineStyle', ':');
xlabel('Time (Seconds)', 'FontSize', 12);
ylabel('Force (N)', 'FontSize', 12);
title('(d) Electromagnetic Force', 'FontSize', 14);
grid on;
xlim([0 0.5]);
ylim([-1.5 0]);

sgtitle('Electromagnet Response to Applied Voltage and Force', 'FontSize', 16,
'FontWeight', 'bold');

%% PLOT 2: Inductance and Power
figure('Position', [100 100 1200 800]);
% Inductance
subplot(2,2,1);
plot(t, L*1000, 'g-', 'LineWidth', 1);
hold on;
xlabel('Time (Seconds)', 'FontSize', 12);
ylabel('Inductance (mH)', 'FontSize', 12);
title('Coil Inductance  $L = k/x$ ', 'FontSize', 14);
grid on;
xlim([0 0.5]);

% Input Power
subplot(2,2,2);
plot(t, p_source, 'b-', 'LineWidth', 1);

```

```
hold on;
xlabel('Time (Seconds)', 'FontSize', 12);
ylabel('Inst. P in r (W)', 'FontSize', 10);
grid on;
xlim([0 0.5]);
ylim([0 3]);

% Resistor Power
subplot(2,2,3);
plot(t, p_resistor, 'b-', 'LineWidth', 1);
hold on;
xlabel('Time (Seconds)', 'FontSize', 12);
ylabel('Inst. input Power (W)', 'FontSize', 10);
grid on;
xlim([0 0.5]);
ylim([0 3.5]);
```